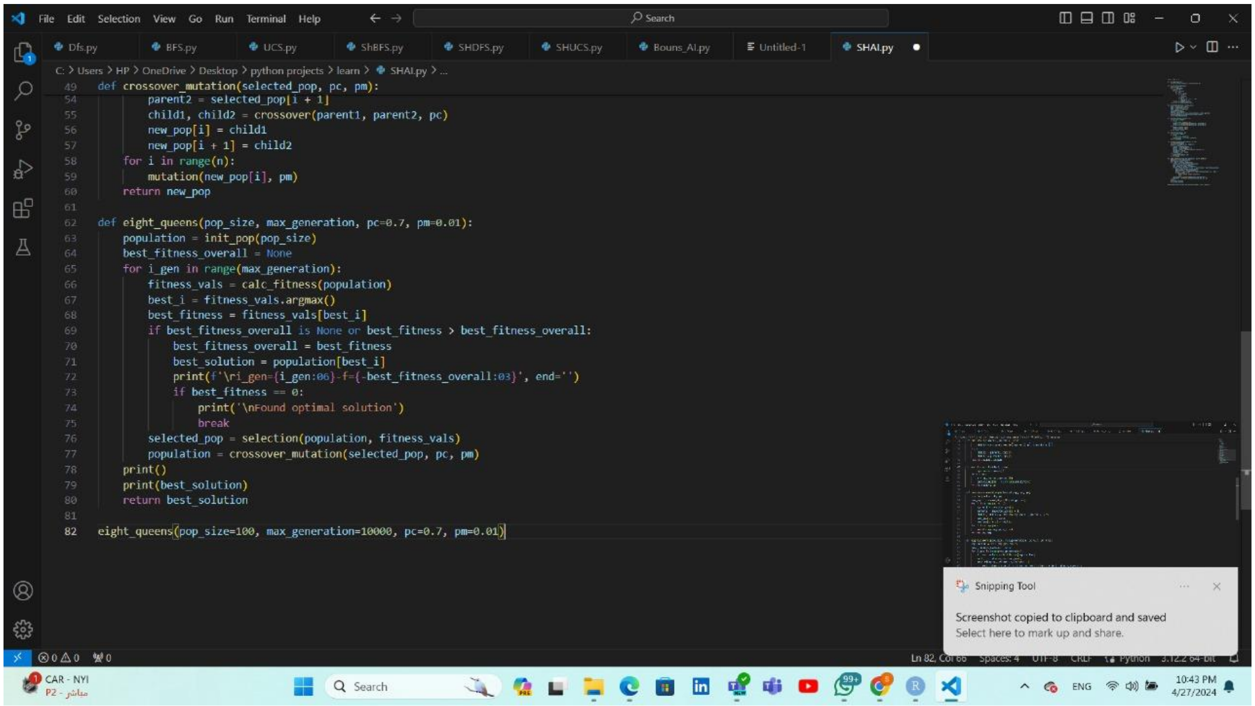
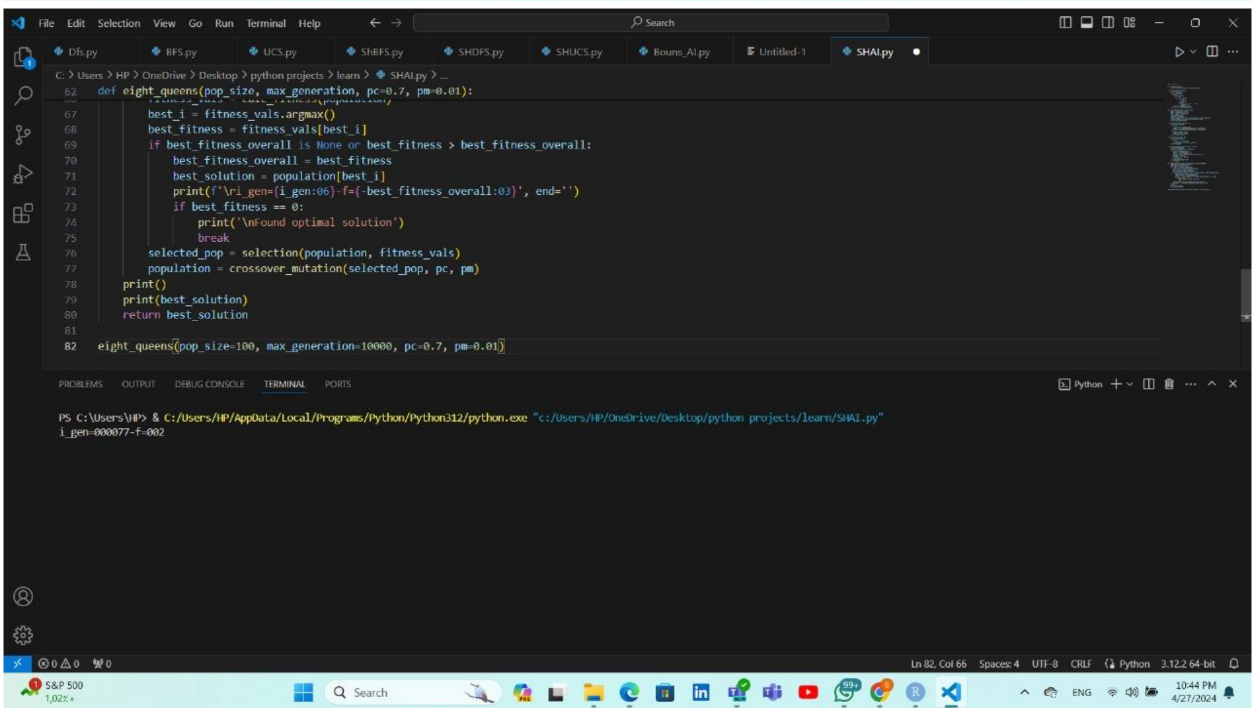
```python
import numpy as np

def init_pop(pop_size):
    return np.random.randint(8, size=(pop_size, 8))

def calc_fitness(population):
    fitness_vals = []
    for x in population:
        penalty = 0
        for i in range(8):
            r = x[i]
            for j in range(8):
                if i == j:
                    continue
                d = abs(i - j)
                if x[j] in [r, r - d, r + d]:
                    penalty += 1
        fitness_vals.append(penalty)
    return -1 * np.array(fitness_vals)

def selection(population, fitness_vals):
    probs = fitness_vals.copy()
    probs += abs(probs.min()) + 1
    probs = probs / probs.sum()
    n = len(population)
    indices = np.arange(n)
    selected_indices = np.random.choice(indices, size=n, p=probs)
    selected_population = population[selected_indices]
    return selected_population

def crossover(parent1, parent2, pc):
    r = np.random.random()
    if r < pc:
        m = np.random.randint(1, 8)
        child1 = np.concatenate([parent1[:m], parent2[m:]])
        child2 = np.concatenate([parent2[:m], parent1[m:]])
    else:
```

```python
def crossover(parent1, parent2, pc):
        child2 = np.concatenate([parent2[:m], parent1[m:]])
    else:
        child1 = parent1.copy()
        child2 = parent2.copy()
    return child1, child2

def mutation(individual, pm):
    r = np.random.random()
    if r < pm:
        m = np.random.randint(8)
        individual[m] = np.random.randint(8)
    return individual

def crossover_mutation(selected_pop, pc, pm):
    n = len(selected_pop)
    new_pop = np.empty((n, 8), dtype=int)
    for i in range(0, n, 2):
        parent1 = selected_pop[i]
        parent2 = selected_pop[i + 1]
        child1, child2 = crossover(parent1, parent2, pc)
        new_pop[i] = child1
        new_pop[i + 1] = child2
    for i in range(n):
        mutation(new_pop[i], pm)
    return new_pop

def eight_queens(pop_size, max_generation, pc=0.7, pm=0.01):
    population = init_pop(pop_size)
    best_fitness_overall = None
    for i_gen in range(max_generation):
        fitness_vals = calc_fitness(population)
        best_i = fitness_vals.argmax()
        best_fitness = fitness_vals[best_i]
        if best_fitness_overall is None or best_fitness > best_fitness_overall:
            best_fitness_overall = best_fitness
            best_solution = population[best_i]
```



```python
def crossover_mutation(selected_pop, pc, pm):
        parent2 = selected_pop[i + 1]
        child1, child2 = crossover(parent1, parent2, pc)
        new_pop[i] = child1
        new_pop[i + 1] = child2
    for i in range(n):
        mutation(new_pop[i], pm)
    return new_pop

def eight_queens(pop_size, max_generation, pc=0.7, pm=0.01):
    population = init_pop(pop_size)
    best_fitness_overall = None
    for i_gen in range(max_generation):
        fitness_vals = calc_fitness(population)
        best_i = fitness_vals.argmax()
        best_fitness = fitness_vals[best_i]
        if best_fitness_overall is None or best_fitness > best_fitness_overall:
            best_fitness_overall = best_fitness
            best_solution = population[best_i]
            print(f'\ri_gen={i_gen:06}-f={-best_fitness_overall:03}', end='')
            if best_fitness == 0:
                print('\nFound optimal solution')
                break
        selected_pop = selection(population, fitness_vals)
        population = crossover_mutation(selected_pop, pc, pm)
    print()
    print(best_solution)
    return best_solution

eight_queens(pop_size=100, max_generation=10000, pc=0.7, pm=0.01)
```



```python
def eight_queens(pop_size, max_generation, pc=0.7, pm=0.01):
        best_i = fitness_vals.argmax()
        best_fitness = fitness_vals[best_i]
        if best_fitness_overall is None or best_fitness > best_fitness_overall:
            best_fitness_overall = best_fitness
            best_solution = population[best_i]
            print(f'\ri_gen={i_gen:06}-f={-best_fitness_overall:03}', end='')
            if best_fitness == 0:
                print('\nFound optimal solution')
                break
        selected_pop = selection(population, fitness_vals)
        population = crossover_mutation(selected_pop, pc, pm)
    print()
    print(best_solution)
    return best_solution

eight_queens(pop_size=100, max_generation=10000, pc=0.7, pm=0.01)
```

```
PS C:\Users\HP> & C:/Users/HP/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/HP/OneDrive/Desktop/python projects/learn/SHAI.py"
i_gen=000077-f=002
```